



Plano de Testes – API ServeRest - Challenge 02

ÍNDICE

- Apresentação
- Objetivo
- Escopo
- Ambiente de Testes
- Análise Inicial
 - Documentos utilizados e anexos:
- Mapa Mental da Aplicação
- Cenários de Teste
- Matriz de Risco
- Matriz de Rastreabilidade
- Visão de Cobertura
 - Cobertura de Requisitos
 - Path Coverage

Apresentação

Este plano tem como objetivo organizar e guiar as atividades de garantia de qualidade da API ServeRest, com foco na cobertura funcional das rotas disponíveis no Swagger, validação de regras de negócio descritas nas User Stories, além de testes exploratórios e fluxos ponta a ponta, simulando o uso real de um e-commerce.

Responsável: @Anna Santoro

Data de criação: 2 de jun. de 2025

Última atualização: 6 de jun. de 2025

Sprint: 6 - Challenge 2

Ferramentas: Postman, Swagger, Confluence, Jira, VSCode, GitHub, Robot Framework.

Objetivo

Garantir que a API ServeRest:

- Atenda aos critérios de aceitação estabelecidos nas User Stories.
- Siga comportamentos RESTful coerentes com o Swagger.
- Esteja funcional e segura para os fluxos principais do negócio.
- Tenha comportamentos esperados diante de cenários negativos, limites e alternativos.
 - Priorização de cenários críticos ao negócio, como login e finalização de compra.

Escopo

- Validação funcional de:
 - /usuarios
 - /login
 - /produtos
 - /carrinhos
- Testes manuais
- Testes exploratórios no endpoint de Carrinho
- Testes automatizados via Robot Framework
- Matriz de Rastreabilidade
- Matriz de Risco

Ambiente de Testes

Item	Valor
------	-------

SO	Windows 11
Navegador	Microsoft Edge 135 (64 bits)
Ferramentas	Postman, Swagger, XMind, Confluence, Jira, Robot Framework
API	ServeRest
Ambiente	Público (homologação)
Tipo de Teste	Funcional, Exploratórios e Automatizados
Período de Execução	5 de jun. de 2025 - 6 de jun. de 2025

Análise Inicial [🔗](#)

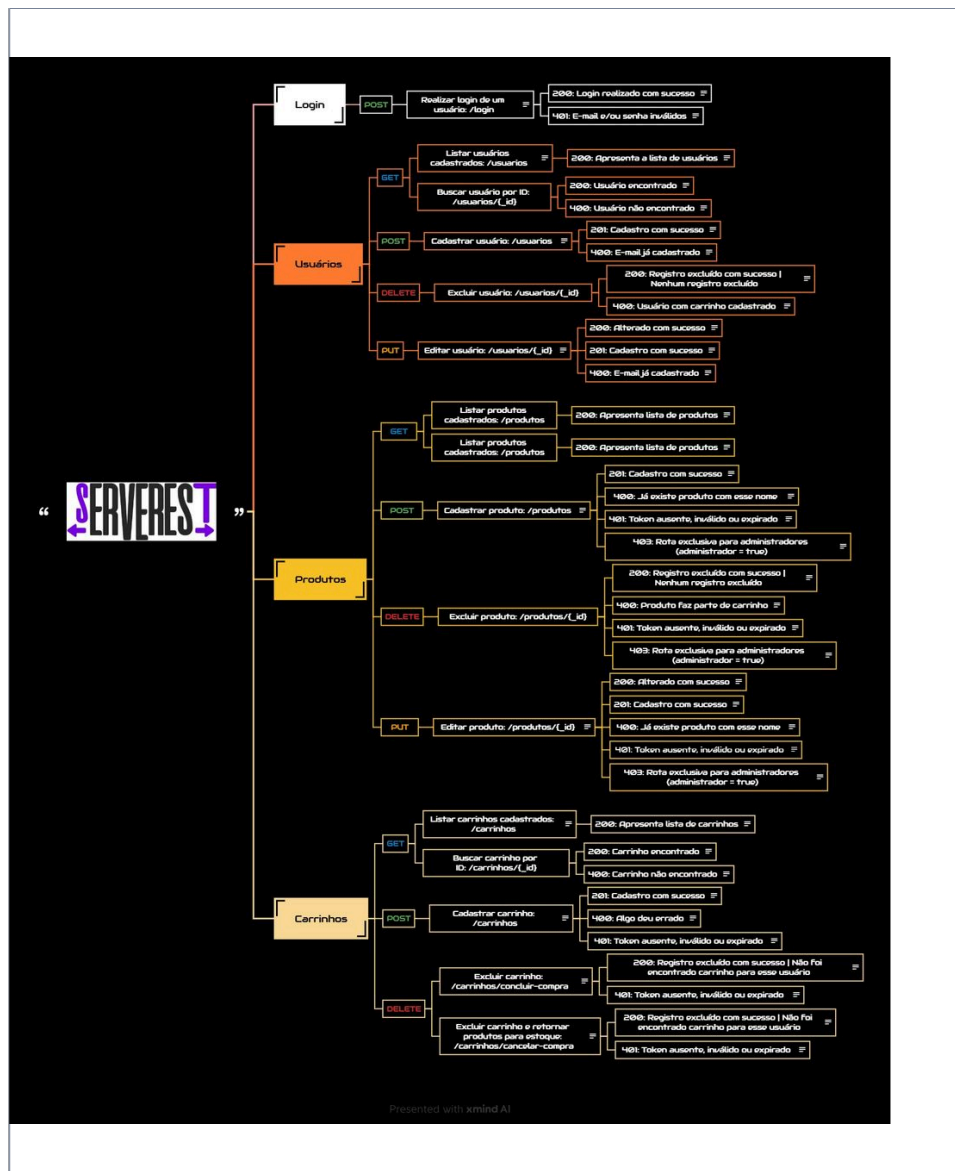
Documentos utilizados e anexos: [🔗](#)

- [Swagger oficial da API ServeRest](#)
- User Stories:
 - [US001 – Usuários](#)
 - [US002 – Login](#)
 - [US003 – Produtos](#)
 - [US004 – Carrinho](#)
- [Mapa Mental](#)
- [Collection Postman](#)
- [Relatório de Testes](#)
- [Issues](#)

Técnicas de Teste Aplicadas	Tipo de Teste	Justificativa
CRUD	Funcional	Base para validar todos os métodos HTTP: <code>POST</code> , <code>GET</code> , <code>PUT</code> , <code>DELETE</code>
Técnicas de Caixa Preta	Funcional	Validar entradas válidas, inválidas e limites críticos.
Goldilocks	Heurística Exploratória	Utilizada nos testes exploratórios no carrinho para validar cenários mínimos, ideais e excessivos

Mapa Mental da Aplicação [🔗](#)

▼ Clique Aqui



Link para mapa extendido: [Clique Aqui](#)

Cenários de Teste

I'D	Título	Pré-condição	Passos	Resultado Esperado	Prioridade	Automação
CT-001	Cadastro válido de novo usuário	E-mail ainda não cadastrado	POST /usuarios com nome, e-mail válido, senha entre 5-10 caracteres, administrador = "true"	201 Created + "Cadastro realizado com sucesso" + ID retornado	CRÍTICA	SIM
CT-002	Cadastro com e-mail duplicado	E-mail já existe na base	POST /usuarios com e-mail já cadastrado	400 Bad Request + "Este email já está sendo usado"	CRÍTICA	SIM
CT-003	Cadastro com e-mail inválido	—	POST /usuarios com e-mail mal formatado (joa@val*d.com)	400 + "email deve ser um email válido"	ALTA	SIM
CT-004	Cadastro com domínio Gmail	—	POST /usuarios com e-mail joao@gmail.com	400 + Mensagem bloqueando domínios Gmail/Hotmail	MÉDIA	SIM
CT-005	Cadastro com senha	—	POST /usuarios com senha de 3 caracteres	400 + "A senha deve conter entre 5 e 10"	ALTA	SIM

	muito curta			caracteres"		
CT-006	Cadastro com senha muito longa	—	POST /usuarios com senha de 11 caracteres	400 + "A senha deve conter entre 5 e 10 caracteres"	ALTA	SIM
CT-007	GET de usuário inexistente	ID não existente	GET /usuarios/{id-invalido}	400 + "Usuário não encontrado"	MÉDIA	SIM
CT-008	PUT criando usuário com ID inexistente	Nenhum usuário com ID fornecido	PUT /usuarios/{id-invalido} com payload de novo usuário	⚠ Ambiguidade	BAIXA	BLOQUE...
CT-009	PUT com e-mail já utilizado	Dois usuários com e-mails diferentes	PUT /usuarios/{id} tentando alterar e-mail para um já existente	400 + "Este email já está sendo usado"	MÉDIA	SIM
CT-010	DELETE de usuário inexistente	ID inválido	DELETE /usuarios/{id-invalido}	200 + "Registro excluído com sucesso Nenhum registro excluído"	MÉDIA	SIM
CT-011	GET de todos os usuários	Existência de usuários na base	GET /usuarios	200 OK + Array de usuários	MÉDIA	SIM
CT-012	Cadastro com campo obrigatório ausente	—	POST /usuarios sem campo "email"	400 + erro por campo obrigatório faltando	ALTA	SIM
CT-013	Valor inválido no campo "administrator"	—	POST /usuarios com "administrator": "talvez"	400 + mensagem de tipo inválido	MÉDIA	SIM
CT-014	Criação de usuário com nome vazio	—	POST /usuarios com "nome": ""	400 + "Nome é obrigatório"	ALTA	SIM
CT-015	Login com credenciais válidas	Usuário válido cadastrado (ex: joao@valido.com , senha 123456)	POST /login com payload válido	200 OK + "Login realizado com sucesso" + Token Bearer retornado	CRÍTICA	SIM
CT-016	Login com usuário não cadastrado	Usuário não existe no sistema	POST /login com e-mail inexistente	401 Unauthorized + "Email e/ou senha inválidos"	CRÍTICA	SIM
CT-017	Login com senha inválida	E-mail válido cadastrado	POST /login com senha errada	401 Unauthorized + Mensagem "Email e/ou senha inválidos"	CRÍTICA	SIM
CT-018	Login sem fornecer senha	—	POST /login com campo "password" vazio ou ausente	400 + Mensagem sobre campo obrigatório	MÉDIA	SIM
CT-019	Login sem fornecer e-mail	—	POST /login com campo "email" vazio ou ausente	400 + Mensagem sobre campo obrigatório	MÉDIA	SIM
CT-020	Verificação de validade do token (duração 10 min)	Usuário logado, token emitido	Esperar 10 minutos e usar o token em endpoint protegido (ex: /produtos)	401 Unauthorized após 10 min, indicando expiração do token	CRÍTICA	SIM
CT-021	Login com token reutilizado após logout	Token válido emitido e logout realizado (se aplicável)	Reutilizar token antigo após logout ou tempo excedido	401 Unauthorized	CRÍTICA	SIM
CT-022	Cadastro de produto com dados válidos	Usuário autenticado (token válido)	POST /produtos com payload: { "nome": "Camiseta Tech", "preco": 99, "descricao": "Camiseta com estampa tecnológica", "quantidade": 10 }	201 Created + "Cadastro realizado com sucesso" + ID retornado	CRÍTICA	SIM

CT-023	Cadastro sem autenticação	Nenhum token fornecido ou token inválido	POST <code>/produtos</code> com payload válido, sem header Authorization	401 Unauthorized + mensagem de autenticação obrigatória	CRÍTICA	SIM
CT-024	Cadastro com nome duplicado	Produto "Camiseta Tech" já cadastrado	POST <code>/produtos</code> com nome já utilizado	400 Bad Request + "Já existe produto com esse nome"	ALTA	SIM
CT-025	Edição com dados válidos	Produto cadastrado + usuário autenticado	PUT <code>/produtos/{id}</code> com alterações válidas no payload	200 OK + "Registro alterado com sucesso"	ALTA	SIM
CT-026	PUT com ID inexistente (criação implícita)	Nenhum produto com ID <code>xyz123</code>	PUT <code>/produtos/xyz123</code> com payload válido	200 ou 201 + mensagem coerente indicando criação	MÉDIA	BLOQUE...
CT-027	Deletar produto sem vínculo	Produto não está em carrinho	DELETE <code>/produtos/{id}</code> com token válido	200 OK + "Registro excluído com sucesso"	ALTA	SIM
CT-028	Deletar produto vinculado a carrinho	Produto em um carrinho ativo	DELETE <code>/produtos/{id}</code>	400 ou 403 + "Não é permitido excluir produto vinculado a um carrinho"	ALTA	SIM
CT-029	Listar todos os produtos	Existem produtos cadastrados	GET <code>/produtos</code>	200 OK + lista de produtos	MÉDIA	SIM
CT-030	Criar produto com campo obrigatório ausente	—	POST <code>/produtos</code> sem o campo "quantidade"	400 Bad Request + mensagem de campo obrigatório ausente	MÉDIA	SIM
CT-031	Criar produto com quantidade negativa	—	POST <code>/produtos</code> com "quantidade": -5	400 Bad Request + mensagem de validação coerente	MÉDIA	SIM
CT-032	Criar produto com nome em branco	—	POST <code>/produtos</code> com "nome": ""	400 Bad Request + mensagem de nome obrigatório	MÉDIA	SIM
CT-033	Criar carrinho com produto válido e quantidade adequada	Usuário autenticado + produto cadastrado	POST <code>/carrinhos</code> com payload válido, ex: { "produtos": [{ "idProduto": "abc123", "quantidade": 2}]	201 Created + "Cadastro realizado com sucesso" + ID do carrinho	CRÍTICA	SIM
CT-034	Criar carrinho com quantidade zero	Usuário autenticado + produto cadastrado	POST <code>/carrinhos</code> com "quantidade": 0	400 Bad Request + mensagem de quantidade inválida	ALTA	SIM
CT-035	Criar carrinho com quantidade acima do estoque	Produto com estoque limitado (ex: 5 unidades)	POST <code>/carrinhos</code> com "quantidade": 9999	400 Bad Request + mensagem de estoque insuficiente	CRÍTICA	SIM
CT-036	Criar carrinho com produto inexistente	Usuário autenticado	POST <code>/carrinhos</code> com <code>idProduto</code> inválido	400 ou 404 + mensagem de erro coerente	CRÍTICA	SIM
CT-037	Deletar carrinho existente	Carrinho criado previamente com produtos	DELETE <code>/carrinhos/{id}</code>	200 OK + "Registro excluído com sucesso"	MÉDIA	SIM
CT-038	Cancelar compra e devolver ao estoque	Carrinho criado e estoque já reduzido	DELETE <code>/carrinhos/cancelar-compra/{id}</code>	200 OK + "Registro excluído com sucesso. Estoque dos produtos restaurado"	ALTA	SIM

CT-039	Criar carrinho sem autenticação	—	POST /carrinhos sem header Authorization	401 Unauthorized + "Token de acesso ausente ou inválido"	CRÍTICA	SIM
------------------------	---------------------------------	---	--	--	---------	-----

Matriz de Risco [↗](#)

ID	Cenário	Impacto	Probabilidade	Risco	Ação de Mitigação
CT-001	Cadastro válido de novo usuário	Alto	Médio	Alto	Teste automatizado contínuo + validação de campos obrigatórios e duplicatas
CT-002	Cadastro com e-mail duplicado	Médio	Alto	Alto	Validação de unicidade no backend + teste de borda automático
CT-003	Cadastro com e-mail inválido	Médio	Médio	Médio	Regex + validação no frontend e backend
CT-005	Senha muito curta	Alto	Médio	Alto	Política de senha mínima + cobertura de testes com valores limite
CT-015	Login com credenciais válidas	Crítico	Médio	Crítico	Teste contínuo automatizado + testes de stress e concorrência
CT-017	Login com senha inválida	Médio	Alto	Alto	Verificação de mensagem de erro + bloqueio após N tentativas
CT-020	Verificar validade do token	Crítico	Médio	Crítico	Automação com simulação de expiração de token e revogação pós-logout
CT-022	Cadastro de produto com dados válidos	Alto	Médio	Alto	Automação com dados diversos + verificação da persistência
CT-024	Cadastro com nome já utilizado	Médio	Alto	Alto	Validação de unicidade + testes com nomes duplicados
CT-025	Edição de produto com dados válidos	Alto	Médio	Alto	Testes automatizados com valores válidos e inválidos
CT-028	Deletar produto vinculado a carrinho	Crítico	Médio	Crítico	Validação em backend + testes de integridade do carrinho após remoção
CT-029	Listar todos os produtos	Médio	Baixo	Médio	Paginação e performance testadas com base de dados grandes
CT-033	Criar carrinho com produto válido e quantidade correta	Crítico	Médio	Crítico	Testes automatizados com diferentes combinações de produto e estoque
CT-034	Criar carrinho com quantidade acima do estoque	Alto	Médio	Alto	Regra de validação + verificação de estoque em tempo real
CT-037	Cancelar compra e devolver produto ao estoque	Crítico	Médio	Crítico	Teste de rollback de estoque + rastreamento de inventário
CT-038	Criar carrinho sem autenticação	Crítico	Médio	Crítico	Validação de autenticação obrigatória + testes de acesso não autorizado

Matriz de Rastreabilidade [↗](#)

US	ID REQ.	Requisito	Cenários de Teste	Prioridade	Criticidade	Status
US001	REQ-001	Usuário deve possuir os campos nome, email, senha, administrador	CT-001, CT-012, CT-013, CT-014	Alta	Alta	EXECUTADO
US001	REQ-002	Não deve permitir ações em usuários inexistentes	CT-007, CT-010	Alta	Alta	EXECUTADO

US001	REQ-003	PUT deve criar usuário se ID não existir	CT-008	Média	Média	BLOQUEADO
US001	REQ-004	Não permitir criação de usuário com e-mail duplicado	CT-002	Alta	Alta	EXECUTADO
US001	REQ-005	PUT não deve permitir criação de usuário	CT-008	Alta	Alta	BLOQUEADO
US001	REQ-006	Bloquear e-mails de domínios gmail/hotmail	CT-004	Alta	Alta	FALHA
US001	REQ-007	Validar formato de e-mail	CT-003	Alta	Alta	EXECUTADO
US001	REQ-008	Senhas devem ter entre 5 e 10 caracteres	CT-005, CT-006	Alta	Alta	FALHA
US002	REQ-009	Usuários não cadastrados não devem se autenticar	CT-016	Alta	Alta	EXECUTADO
US002	REQ-010	Senha inválida impede autenticação	CT-017	Alta	Alta	EXECUTADO
US002	REQ-011	401 em falha de autenticação	CT-016, CT-017	Alta	Alta	EXECUTADO
US002	REQ-012	Autenticação válida gera token	CT-015	Alta	Alta	EXECUTADO
US002	REQ-013	Token deve durar 10 minutos	CT-020	Alta	Crítica	EXECUTADO
US002	REQ-014	Rejeitar token expirado ou inválido	CT-021	Alta	Alta	NÃO INICIA...
US002	REQ-015	Campos obrigatórios (email e password) devem ser validados	CT-018, CT-019	Média	Alta	EXECUTADO
US003	REQ-016	Requer autenticação para ações em produtos	CT-023	Alta	Alta	NÃO INICIA...
US003	REQ-017	Não permitir cadastro de produtos com nome duplicado	CT-024	Alta	Alta	NÃO INICIA...
US003	REQ-018	Não excluir produtos em carrinho	CT-028	Alta	Alta	NÃO INICIA...
US003	REQ-019	PUT cria produto se ID não existir	CT-026	Média	Média	BLOQUEADO
US003	REQ-020	PUT não pode reutilizar nomes	CT-024, CT-025	Média	Média	NÃO INICIA...

● Azul = Não iniciado | ● Vermelho = Falha detectada | ● Verde = Executado (sucesso) | ● Amarelo = Bloqueado

Visão de Cobertura [🔗](#)

Cobertura de Requisitos [🔗](#)

- Total de Requisitos: **20**
- Cobertos: **20**
- Cobertura Geral: **100%**

Path Coverage [🔗](#)

- Endpoints totais: **9**
- Cobertos por testes automatizados: **8**
- Path Coverage: **88,89%**

• Total de cenários indicados para automação: 39

• Sugerido uso do Postman + Tests Tab + Newman Runner